武 汉 工 程 大 学

计算机科学与工程学院

《操作系统》实验报告

专业班级	人工智能 01 班	实验地点	503
学生学号	2321050119	指导教师	熊向群
学生姓名	李枝	实验时间	11. 17
实验项目	实验一、进程控制与通信		
实验类别	操作性（） 基础性（ √ ） 设计性（） 综合性（ ） 其它		
实验目的及要求	1、掌握 Linux 或 Windows 关于进程创建与进程通信的原理。 2、研究 Linux 或 Windows 的进程创建与进程控制机制。 3、编程实现进程创建与进程三种通信方式。		

成 绩 评 定 表

类 别	评 分 标 准	分值	得分	合 计
实验结果	完成及时，程序通畅 功能齐全，界面友好	40 分		
实验报告	及时递交、填写规范 内容完整、体现收获	60 分		

评语：

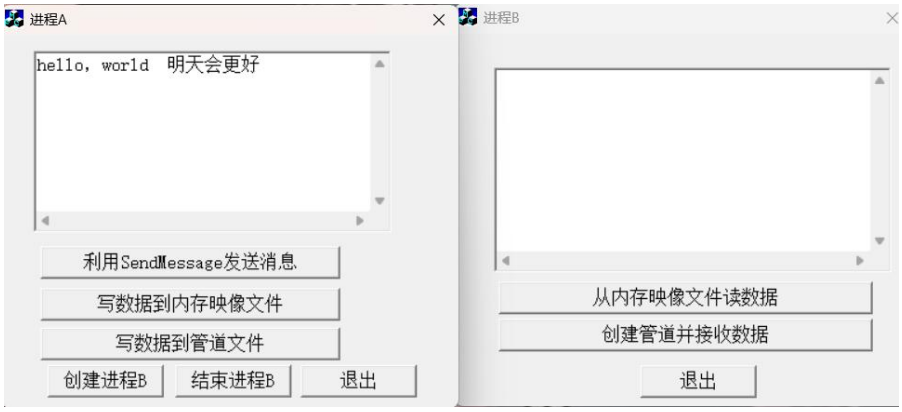
实 验 内 容

一、实验内容

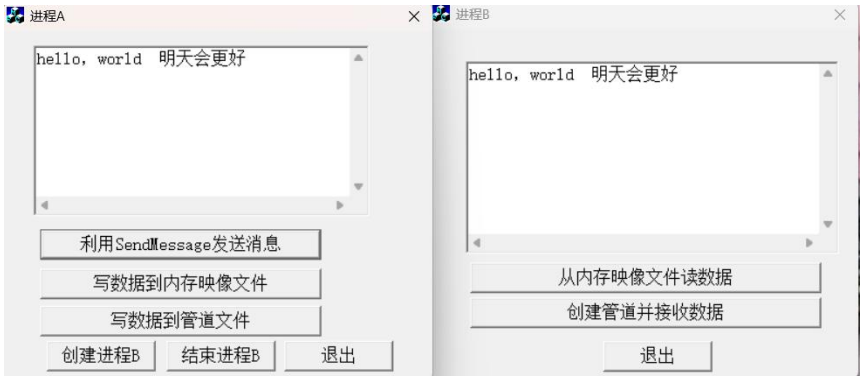
目的：创建进程，实现消息通信和共享内存通信。
进程创建的核心是操作系统分配资源（内存、PID 等）并加载程序代码，进程通信（IPC）则是通过内核提供的 “中间载体”（消息队列、共享内存等）实现数据交换。进程是操作系统资源分配的基本单位，创建本质是 “复制 / 初始化资源 + 加载执行程序” 的过程。
Windows 进程创建
关键步骤：
操作系统为新进程分配 PID、虚拟地址空间、句柄表（管理文件、内存等资源）。
加载器读取可执行文件（.exe），解析 PE 格式，加载依赖的 DLL，初始化进程环境（全局变量、堆等）。
创建主线程，从程序入口点（如 WinMain）开始执行。
特点：进程与线程绑定更紧密，创建时默认包含一个主线程，资源分配粒度更细（如私有堆、线程局部存储）。
进程地址空间相互独立（用户态隔离），无法直接访问对方数据，需通过内核提供的 “共享载体” 或 “转发机制” 实现通信，不同 IPC 方式的核心差异在于 “载体类型” 和 “数据传输效率”。
核心 IPC 机制原理：
消息通信：
内核维护一个 “消息队列”，发送方将数据打包成 “消息” 存入队列，接收方从队列中读取，内核负责同步（避免读写冲突）。
共享内存：
内核分配一块物理内存，映射到多个进程的虚拟地址空间，进程直接读写该内存（无需内核转发），效率最高。
管道通信：
内核创建一个 “伪文件”（管道），采用 “先进先出”（FIFO）模式，进程通过文件读写接口（read/write）交换数据，仅支持单向通信。
信号量：
内核维护一个计数器，用于进程间同步（如控制共享资源的访问权限），不直接传输数据，常与共享内存配合使用。

三种通信方式：
匿名管道：父子进程间单向通信。
命名管道：无亲缘关系进程间通信（服务端 + 客户端）。
共享内存：高效数据传输（写端 + 读端）。

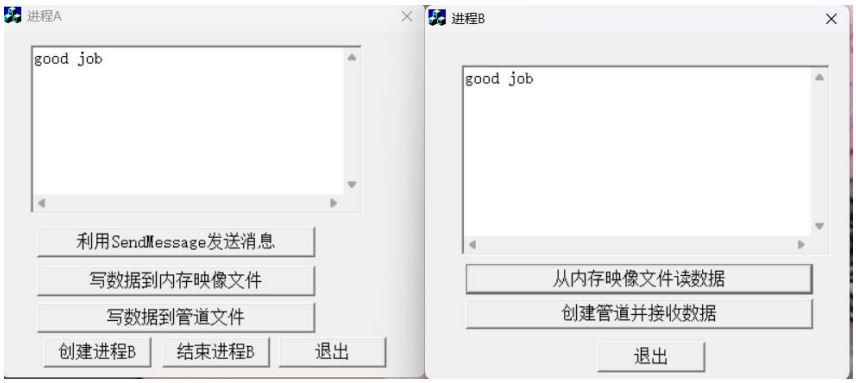
实验结果图：



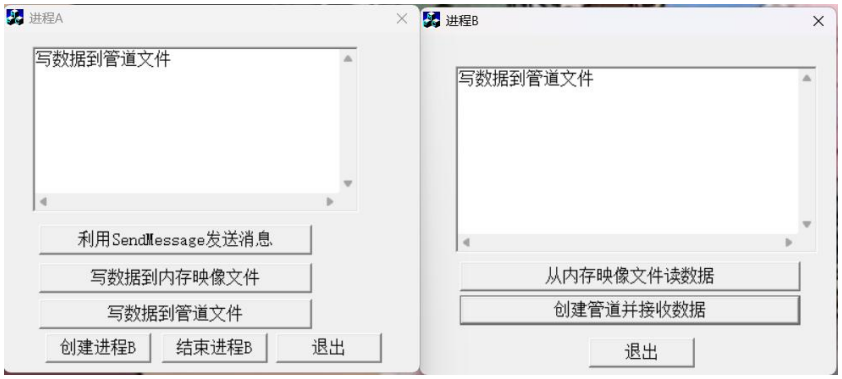
利用 sendmessage 发送消息：



写数据到内存映像文件和从内存映像文件读数据：



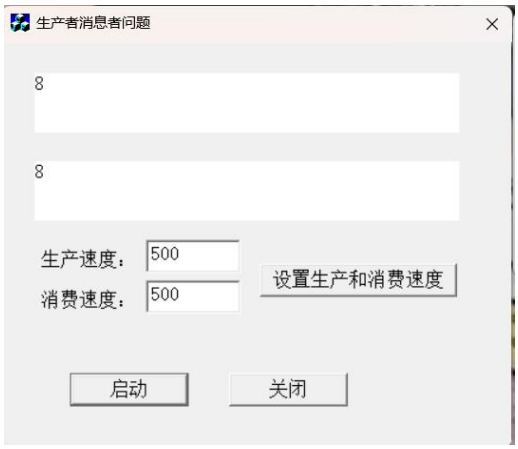
写数据到管道文件以及接收：



创建进程 B:



生产者与消费者问题：



创建线程：



线程互斥：



非互斥运行：



结束线程：



二、实验总结

本次实验围绕进程创建、进程通信与线程同步展开，通过实践深入理解了操作系统中并发编程的核心机制。

在进程部分，我们掌握了 Windows 平台下CreateProcess的使用，明确了程序路径、权限配置对进程创建的影响，解决了 “错误码 2” 等路径问题。三种进程通信方式的实现让我认识到：匿名管道仅适用于父子进程，命名管道支持无亲缘关系进程甚至跨网络通信，而共享内存凭借直接内存映射成为效率最高的 IPC 方式，不过需手动同步以避免冲突。

线程同步实验中，生产者 - 消费者与线程互斥案例直观展现了同步原语的价值。通过信号量、互斥锁的应用，我们成功解决了缓冲区溢出 / 空转、共享资源竞争等问题，理解了 “互斥保证资源独占，同步协调执行顺序” 的设计逻辑。对比非互斥运行时的混乱结果，更深刻体会到同步机制对并发程序稳定性的关键作用。

实验也暴露了我在初期对 API 参数、权限管理的生疏，经过反复调试才掌握进程创建的路径规范与线程同步的逻辑设计。此次实践不仅巩固了进程线程的理论知识，更锻炼了排查系统 API 错误、设计并发同步逻辑的能力，为后续复杂并发程序开发奠定了基础。

附录代码：
匿名管道代码：
父进程代码（写端）：

```
#include <stdio.h>#include <windows.h>
#define BUFFER_SIZE 256
void main(void) {
    HANDLE hReadPipe, hWritePipe;  // 管道读端、写端句柄
    SECURITY_ATTRIBUTES sa;        // 安全属性（控制句柄继承）
    PROCESS_INFORMATION pi;        // 子进程信息
    STARTUPINFO si;                // 子进程启动信息
    DWORD BytesWritten;            // 实际写入字节数
    char Buffer[] = "This is a test via anonymous pipe";  // 待发送数
据

    // 初始化安全属性：允许句柄继承（匿名管道核心）
    sa.nLength = sizeof(SECURITY_ATTRIBUTES);
    sa.lpSecurityDescriptor = NULL;
    sa.bInheritHandle = TRUE;  // 关键：子进程可继承管道句柄

    // 创建匿名管道
    if (!CreatePipe(&hReadPipe, &hWritePipe, &sa, 0)) {
        printf("CreatePipe failed with error %d\n", GetLastError());
        return;
    }

    // 初始化子进程启动信息
    ZeroMemory(&si, sizeof(STARTUPINFO));
    si.cb = sizeof(STARTUPINFO);
    si.hStdInput = hReadPipe;  // 子进程标准输入重定向到管道读端
    si.dwFlags = STARTF_USESTDHANDLES;  // 使用自定义标准输入

    // 创建子进程（启动 cmd.exe 作为读端示例）
    if (!CreateProcess(
        "C:\\Windows\\system32\\cmd.exe",  // 子进程路径（系统自带程
序）
        NULL, NULL, NULL,
        TRUE,  // 允许继承句柄（与 sa.bInheritHandle 配合）
        0, NULL, NULL,
        &si, &pi
    )) {
        printf("CreateProcess failed with error %d\n", GetLastError());
        CloseHandle(hReadPipe);
        CloseHandle(hWritePipe);
        return;
    }
```

```
    // 向管道写数据（父进程 → 子进程）
    if (!WriteFile(hWritePipe, Buffer, sizeof(Buffer), &BytesWritten,
NULL)) {
        printf("WriteFile failed with error %d\n", GetLastError());
        CloseHandle(hReadPipe);
        CloseHandle(hWritePipe);
        CloseHandle(pi.hProcess);
        CloseHandle(pi.hThread);
        return;
    }

    printf("Parent wrote %d bytes: %s\n", BytesWritten, Buffer);

    // 清理资源
    CloseHandle(hReadPipe);
    CloseHandle(hWritePipe);
    CloseHandle(pi.hProcess);
    CloseHandle(pi.hThread);}
```

子进程代码：

```
#include <stdio.h>#include <windows.h>
#define BUFFER_SIZE 256
void main(void) {
    HANDLE hReadPipe;  // 管道读端句柄（继承自父进程）
    DWORD BytesRead;   // 实际读取字节数
    char Buffer[BUFFER_SIZE];  // 接收缓冲区

    // 获取继承的管道读端句柄（父进程已重定向标准输入为管道）
    hReadPipe = GetStdHandle(STD_INPUT_HANDLE);
    if (hReadPipe == INVALID_HANDLE_VALUE) {
        printf("GetStdHandle failed with error %d\n", GetLastError());
        return;
    }

    // 从管道读取数据（父进程发送的数据）
    if (!ReadFile(hReadPipe, Buffer, BUFFER_SIZE, &BytesRead, NULL)) {
        printf("ReadFile failed with error %d\n", GetLastError());
        CloseHandle(hReadPipe);
        return;
    }

    printf("Child read %d bytes: %s\n", BytesRead, Buffer);

    // 清理资源
    CloseHandle(hReadPipe);}
```

命名管道代码：
客户端：

```
#include <stdio.h>
#define PIPE_NAME "\\\\cqu-zlh\\Pipe\\zhj"
void main(void) {
    HANDLE PipeHandle;
    DWORD BytesWritten;
    if (WaitNamedPipe(PIPE_NAME, NMPWAIT_WAIT_FOREVER) == 0)
    {
        printf("WaitNamedPipe failed with error %d\n",
            GetLastError());
        return;
    }
    // Create the named pipe file handle
    if ((PipeHandle = CreateFile(PIPE_NAME,
        GENERIC_READ | GENERIC_WRITE, 0,
        (LPSECURITY_ATTRIBUTES) NULL, OPEN_EXISTING,
        FILE_ATTRIBUTE_NORMAL,
        (HANDLE) NULL)) == INVALID_HANDLE_VALUE)
    {
        printf("CreateFile failed with error %d\n", GetLastError());
        return;
    }

    if (WriteFile(PipeHandle, "This is a test", 14, &BytesWritten,
        NULL) == 0)
    {
        printf("WriteFile failed with error %d\n", GetLastError());
        CloseHandle(PipeHandle);
        return;
    }

    printf("Wrote %d bytes", BytesWritten);

    CloseHandle(PipeHandle);
}
```

服务器端：

```
DWORD WINAPI PipeInstanceProc(LPVOID lpParameter)
{
    HANDLE PipeHandle;
    DWORD BytesRead;
    DWORD BytesWritten;
```

```
CHAR Buffer[256];
static int i;

if ((PipeHandle = CreateNamedPipe("\\\\.\\PIPE\\jim",
   PIPE_ACCESS_DUPLEX, PIPE_TYPE_BYTE | PIPE_READMODE_BYTE,
   NUM_PIPES, 1024, 1024, 2000, NULL)) == INVALID_HANDLE_VALUE)
{
   printf("CreateNamedPipe failed with error %d\n",
      GetLastError());
   return 0;
}

// Serve client connections forever
while(1)
{
   if (ConnectNamedPipe(PipeHandle, NULL) == 0)
   {
      printf("ConnectNamedPipe failed with error %d\n",
         GetLastError());
      break;
   }

   // Read data from and echo data to the client until
   // the client is ready to stop
   while(ReadFile(PipeHandle, Buffer, sizeof(Buffer),
      &BytesRead,  NULL) > 0)
   {
      printf("Echo %d bytes to client\n", BytesRead);
       printf("the thread%d", i++);
      if (WriteFile(PipeHandle, Buffer, BytesRead,
         &BytesWritten, NULL) == 0)
      {
         printf("WriteFile failed with error %d\n",
            GetLastError());
         break;
      }
   }

   if (DisconnectNamedPipe(PipeHandle) == 0)
   {
      printf("DisconnectNamedPipe failed with error %d\n",
         GetLastError());
      break;
   }
```

```
    }

    CloseHandle(PipeHandle);
    return 0;
}
```

共享内存通信代码：

```
    //创建内存映像对象
        HANDLE hMapping;
        LPSTR lpData;
        hMapping=CreateFileMapping((HANDLE)0xFFFFFFFF,NULL,PAGE_READWR
ITE,0,0x200,"MYSHARE");
        if(hMapping==NULL)
        {
            AfxMessageBox("CreateFileMapping() failed.");
            return;
        }
        //将文件的视图映射到一个进程的地址空间上，返回 LPVOID 类型的内存
指针
        lpData=(LPSTR)MapViewOfFile(hMapping,FILE_MAP_ALL_ACCESS,0,0,0
);
        if(lpData==NULL)
        {
            AfxMessageBox("MapViewOfFile() failed.");
            return;
        }
        //给这段映像内存写数据
        sprintf(lpData,m_Msg1);
        //释放映像内存
    UnmapViewOfFile(lpData);
```